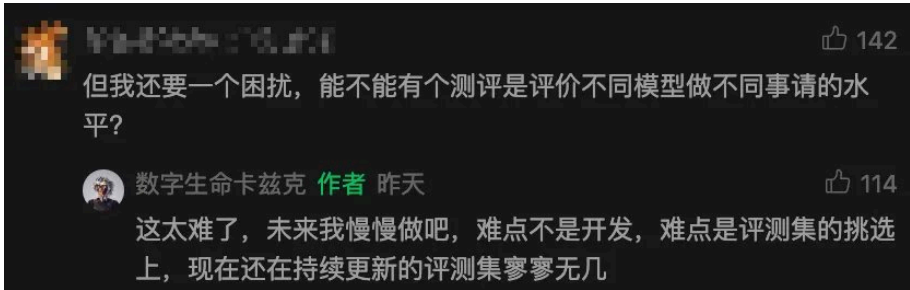


摘要：

据评测，当前主流大模型在真实电商、生活等场景任务中表现远逊于编程领域。例如，在RealReplicaBench电商评测中，仅Claude Opus 5通过率刚过60%，多数模型不足50%。在AI的世界里，会解题，和会工作，中间确实还隔着一些比较遥远的距离。

上周一，我发了自己做的AIHOT大模型排行榜。

有很多朋友感兴趣，在文章下面评论，最高赞的一条，是在问能不能有个分类测评，能够评估不同模型做不同任务的表现。



讲道理，这其实一直是我想做的，也是我觉得特别特别核心的痛点。

现在的大模型榜单很多，比如LiveBench、DeepSWE之类的，可以很快知道哪个模型在某一个能力维度上更强，像Coding、数学、浏览器操作等等。

但是有一类评测集，其实是最稀缺的。

就是模型在真实工作中的实际完成效果。

比如金融领域、法律领域、电商领域、生活领域等等等等，就是直接给它一份真实工作，让它自己从头干到尾，最终看工作效果，而这些，我觉得其实才更符合真实用户的日常工作和生活，也对大家更有用。

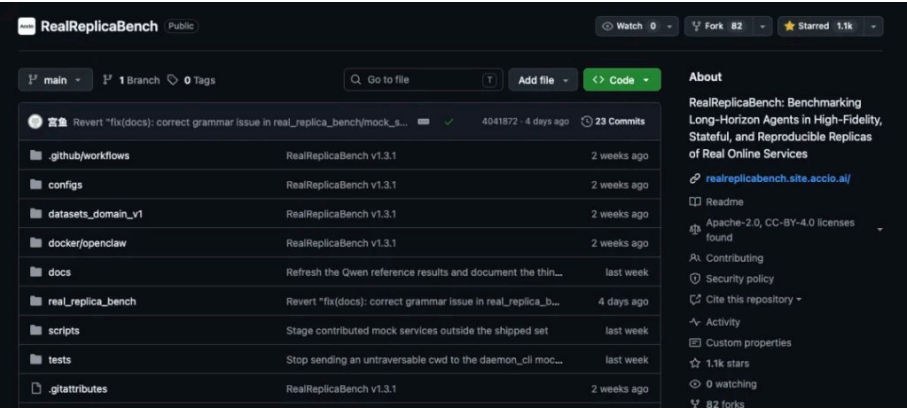
所以，在分类上，我一直想把这些场景给补上，把这个排行榜的维度，变得更加丰富和多维化。

于是，这个周末，我几乎把所有的评测集都翻了一遍，不翻不知道，一翻才吓了一跳，这块的评测集，真的少的太可怜的，真的可以用寥寥无几来形容。

而且绝大多数的模型的跑分，在这些真实场景下任务的成功率，也是真的低的可怜，跟大家在Coding领域的体感完全不一样。

我用一个周末我翻到的比较成熟的电商评测的评测集来举例子，正好也是上周开源的，比较新，一些最新的模型也都有。

这个评测集叫RealReplicaBench，团队来自阿里国际站，因为他们就是做外贸的，所以内部的数据我觉得还是比较权威的，从这个案例，也可以看看Agent在真实的电商场景下能完成什么样子。



这套评测一共有107个任务，阿里国际站因为直接做了Accio Work这个电商Agent，然后他们就从大概过去160万条完整对话中整理出20万条工作流，再归纳出约2000条商业价值高的，最后按照可复现性和结果可判定性，最终整理出来107个任务。

REALREPLICABENCH · V1.3.1

107 项任务速览

4 类操作入口 · 29 个平台 / 任务域 · 数量由任务目录复算，并与官方 collection 逐项一致

● API / MCP

结构化接口调用、跨系统状态变更

10

Amazon

毛利审计 / 改价发布

1

Gmail

简报 / 发票 / RFQ

3

Google Docs

风险汇总 / 价表更新

2

Slack

报价整合与推荐

1

Slack + Contacts

异常路由 / 状态简报

2

Travel

供应商行程交接

1

● Browser

网页多步操作、视觉定位与状态闭环

28

Alibaba

商品发布 / 店铺诊断

7

Freightos

订舱 / 路线优化

4

Gmail

邮件研判 / 装运预约

7

Google Docs

评论闭环 / Feed QA

2

Mail + Calendar

供应商会议协调

1

Reddit

论坛调研

1

Shopify

商品变体 / 主题焕新

3

Web Research

市场 / 竞品 / Listing

3

● CLI

命令行操作与跨应用长链路编排

53

Alibaba

多源商品发布

6

Amazon

采购候选筛选

1

Box

文档合规 / 资料清理

2

Commerce

上架 / 大促 / 对账

5

DWS Docs

文档编辑 / 合规 QA

6

Google Trends

数据与报告审计

3

Google Workspace

补货 / 核对 / 报价

3

Jira

缺陷分诊与排期

1

Notion

Backlog / 证据 / 发布

3

Stripe

账单 / 订阅 / 支付

8

Todoist

采购能力分诊

1

Cross-app Xfer

召回 / 海关 / 采购塔

14

● File

生成、修订和审计最终交付文件

16

Commerce Ops

网站 / 图表 / 运营方案

10

Google Trends

CSV / 比较 / 报告审计

4

Logistics

货运保险 / 危险品

2

模型输入要求

65 纯文本 · 20 浏览器文本 · 22 必须视觉

口径：官方目录分类：main @ 4041872，核对于 2026-08-16。

基本把电商领域涉及到的工作场景都覆盖了，像供应商采购、商品发布、店铺经营、订单对账、国际物流、售后争议等等等等。

任务具体内容的分布我让GPT做了张图，方便大家更直观的看一下。



模型拿到的信息有发品计划、三家供应商的报价和一堆实拍图片。

然后呢，它要找出计划指定的供应商，填好类目、销售国家、差异定价和描述，再从一堆相机自动编号的照片里挑出正确、清晰的商品图上传。

还有一个，是给东莞到达拉斯的消费级电子产品规划运输路线。

帮我给一票东莞到 Dallas 的 40' FCL 消费电子货物，规划一条 30 天内总成本最低的多段路线，并在货代订舱平台上完成海运段的 booking 和 shipment verification。平台开在 <http://127.0.0.1:3003/login>，[登录进去操作](#)，[账号密码在 workspace/portal_credentials.txt](#)。

这是多段运输（国内拖车 + 海运 + 美国尾程），各段之间的中转港口必须衔接得上。货物信息、各段成本的算法和约束都在 workspace/shipping_requirements.md，按它来：枚举所有可行的路线组合（港口能对接、总运输时间 ≤ 30 天），在里面挑总成本最低的那条——总成本要算上 insurance、customs clearance、customs bond、platform fee 这些全部费用。海运段下单后完成 shipment verification，发货方 / 收货方信息填 workspace/company_details.md。

把过程和结果写到 outputs/route_analysis.md：路线分析过程、各组合的可行性判断与成本明细、最优路线的选择理由，以及最终 booking 信息（carrier、quote、shipment ID）。

在30天的运输时间的限制下，模型需要计算保险、清关、海关担保和平台费，找出总成本最低的路线，再在仿真的货代平台里完成海运段订舱和货件验证。

还有处理电商退货争议。

我正在为一批电商平台的退货案例整理可辩护的退货解决方案证据包。对于每笔退货，请决定卖家是否应该接受退货、部分退款、全额退款、提出争议/请求平台审核，或者要求提供更多证据——并以订单事件、承运商扫描记录、检验记录、服务水平协议 (SLA) 截止日期、客户消息、本地照片以及书面政策为依据做出决定。全程保持中立：不要指责买家，使用规范的语言，重点关注证据、政策适用性和后续行动。

所有数据都在本地 workspace/，这是一项离线任务。退货中心导出的 returns/return_center_export.csv 数据仅为批次索引——它本身包含的信息不足以支持任何决策。每个案例的详细信息存在于其他地方：订单事件在[此处应填写订单事件列表] orders/order_event_log.jsonl，承运商扫描信息在[此处应填写承运商扫描信息列表] carrier/carrier_scan_history.json，RMA检验结果在[此处应填写RMA检验结果列表] inspection/rma_inspection_worksheet.csv，客户对话记录在[此处应填写客户对话记录列表] messages/customer_thread.md，包裹/标签/退货商品照片在[此处应填写包裹/标签/退货商品照片列表] photos/，平台退货政策和卖家退货标准操作程序在 policies/ [此处应填写平台退货政策和卖家退货标准操作程序列表 marketplace_return_policy.pdf] seller_sop_returns.md，以及案例截止日期在[此处应填写案例截止日期 sla/marketplace_case_deadlines.csv 列表]。为了使每个决策都站得住脚，您必须实际阅读相关的案例资料，并引用您使用的具体资料。

将三个文件分别写入 outputs/：return_decision.json、evidence_pack.md 和 customer_safe_response.md。

return_decision.json 使用这种形状：

```
{
  "case_decisions": [
    {
      "return_id": "RET-EXAMPLE",
      "order_id": "ORD-EXAMPLE",
      "decision": "accept_return | partial_refund | full_refund | dispute_claim | request_more_evidence",
      "decision_reason": "short explanation referencing the policy clause and the key factual finding",
      "evidence_items": [
        {
          "evidence_id": "E1",
          "source_file": "workspace/<subdir>/<filename>",
          "supports": "specific factual datum quoted/paraphrased from this exact file",
          "confidence": "high | medium | low"
        }
      ],
      "policy_cites": ["page or section references from marketplace_return_policy.pdf or seller_sop_returns.md"],
      "sla_or_followup_notes": ["per-case marketplace_case_id and/or sla_hours_remaining from sla/marketplace_case_deadlines.csv"],
      "next_actions": ["operational actions"]
    }
  ],
  "batch_notes": ["cross-case observations"]
}
```

这个任务里面模型需要把订单记录、物流扫描、仓库质检、客户对话、现场照片、平台政策和截止时间全对起来，然后逐单决定接受退货、部分退款、全额退款、向平台申诉，还是继续补充证据。

甚至还有跨平台月度对账。

我是 LuminaCare (露美薇) 的运营数据分析师, 品牌在天猫、京东、拼多多三个平台开旗舰店, 卖 5 款核心护肤品。月初要做 6 月份的月度对账, 所有数据已经导到 workspace/ 下了。先看 workspace/task_brief.md 了解数据文件清单和整体工作, 对账业务规则 (状态映射、历史编码、佣金口径、结算时间差、异常判定) 写在 workspace/reference/business_rules.md 里, 动手前认真读一遍。

把对账结果写到 outputs/ 下 4 个文件:

outputs/unified_orders.csv

三个平台的订单数据标准化合并成一张表, 列严格按这个顺序:

```
order_id,platform,internal_sku,product_name,qty,paid_amount,order_date,status
```

- platform 取值 tmall / jd / pdd
- internal_sku 通过 reference/sku_master.csv 映射, 天猫的遗留编码按 business_rules.md 第 1.3 节的历史编码对照表处理
- product_name 用 sku_master.csv 的 product_name
- order_date 统一为 yyyy-mm-dd
- status 统一为英文 completed / cancelled / refunded / pending / paid / shipped, 映射见 business_rules.md
- 拼多多的数据先按 trade_id 去重
- paid_amount 保留 2 位小数
- 不要表头

outputs/exceptions.json

异常报告, 三个部分:

```
{  "fulfillment_exceptions": [...],  "settlement_discrepancies": [...],  "data_quality_issues": [...]}
```

fulfillment_exceptions: 只检查状态 completed 的订单, 把 qty 和 shipment_log.csv 里对应 platform_order_id 的 qty_shipped 合计比对。每条:

它要把天猫、京东、拼多多三套格式不同的原始订单清洗到一张表里, 统一SKU和订单状态, 核对发货与结算差异, 再算出每个SKU在各个平台的收入、成本、佣金和利润, 最后把这些信息按照指定格式进行交付。

从这几个例子就能看到, 这基本都是现实世界的电商交易的真实任务, 商品上架、物流规划, 到售后和经营分析, 基本全都有, 而且讲道理, 这些任务看着就是脏活累活, 理论上都应该交给Agent干对吧。

然后呢, 我们来看看, 现在在Coding上感觉已经大杀四方, 感觉什么都能干出来的这些牛逼模型, 在这个真实任务场景上的成功率有多少。

#	模型	通过率	通过数	每任务成本	每任务耗时
1	Claude Opus 5	60.75%	65 / 107	\$1.79	7m 48s
2	Claude Opus 4.8	52.34%	56 / 107	\$1.83	8m 42s
3	DeepSeek V4 Pro	49.53%	53 / 107	\$0.201	10m 34s
4	Qwen 3.8 Max	49.53%	53 / 107	\$0.721	13m 53s
5	Claude Opus 4.7	48.60%	52 / 107	\$1.86	6m 53s
6	GPT-5.6 Sol	48.60%	52 / 107	\$1.08	5m 27s
7	OLM 5.2	46.73%	50 / 107	\$0.281	15m 54s
8	Gemini 3.6 Flash	43.93%	47 / 107	\$2.03	8m 52s
9	GPT-5.5	43.93%	47 / 107	\$1.31	6m 58s
10	DeepSeek V4 Flash	42.99%	46 / 107	\$0.064	6m 15s
11	Gemini 3.5 Flash	41.12%	44 / 107	\$3.27	13m 31s
12	GPT-5.6 Luna	39.25%	42 / 107	\$0.029	4m 39s
13	Gemini 3.5 Flash	30.87%	33 / 107	\$0.610	6m 30s

这些模型, 是在PI这个开源Harness框架下跑的, 所以准确性我觉得没啥问题。

百分制下, 只有排名第一的Claude Opus 5刚刚过了及格线。

107个任务, 它完成了65个, 通过率60.75%。

Qwen 3.8 Max和DeepSeek V4 Pro以49.53%的通过率并列第三。

也就是说, 现在大部分模型的任务通过率甚至连50%都过不去, 107个任务, 你让AI去做, 有一半是全失败的。



这就是现在的模型，在真实世界工作中，非常真实的现状。

我也详细看了看他们是怎么去衡量模型到底完没完成，以及给不给分的，看完以后，我感觉比我想象的还要复杂一些。

我随机挑一道供应商采购题，来给大家看看评测过程。

我是 **NorthBridge Accessories** 的采购经理 **Dana Whitfield**。我这几天不在，Gmail 邮箱里堆满了邮件。正在进行的采购项目——“阿特拉斯项目”（**Project Atlas**），一个**阳极氧化铝笔记本电脑支架**的询价——今天必须推进：项目简介、最新规格、数量以及所有供应商的报价都堆在我的邮箱里，供应商启动会议也即将召开。

我的 Gmail 只能通过 `http://127.0.0.1:3071` 访问，使用类似 Google Workspace 的工具接口——使用 `GET /api/workspace/tools` 获取工具列表/输入架构，使用 `POST /api/workspace/call` 并传入 `{ "name": "<tool>", "arguments": { ... } }` 来调用工具。GET `GET /health` 用于检查邮箱状态。所有邮箱的读写操作都使用此接口。无需外部网络。

`workspace/` 设置规则中有两份参考文档——请先阅读它们：

- `sourcing_brief.md` — 我需要生成的内容、对账规则、供应商身份/去重规则和财务安全策略。
- `landed_cost_reference.md` — 包含货币换算、运费/关税数据、国际贸易术语解释通则规范化规则以及供应商选择规则。由于报价采用不同的国际贸易术语解释通则和**货币**，我必须将它们全部规范化为单位美元到岸成本。

询价单详情和报价**不在**工作区，而是在**收件箱**中，分散在许多互不相关的内部邮件中。报价单的显示名称/地址/域名也各不相同。请严格遵循 `sourcing_brief.md` 文件中关于不断变化的需求和供应商身份识别的说明；**不要**仅凭最后一次提及的数值就下结论。也**不要**轻信公司名称或域名。

使用这两份参考文件核对收件箱中的 Project Atlas RFQ，然后在解决变体标识、被取代的报价、撤回、经纪人转发的报价和相似的供应商之后，按标准化的单位美元到岸成本比较供应商。

一旦确定了供应商，请将邮箱和日历设置为 Dana 为即将召开的启动会议所需的状态：将选定供应商的约束性报价**准确**标记为 `RFQ-Decision`，将财务安全例外信息**准确**标记为 `Security-Review`，在 RFQ 简报中列出的启动日期创建供应商启动会议日历事件，并将发送给选定供应商的草稿回复保存到约束性报价单上显示的电子邮件地址。请勿发送邮件。

将 `outputs/triage_summary.json` 写入最终摘要工件，遵循 `sourcing_brief.md` 中的确切模式，包括 `allowed_lead_days`、亚军、`flagged_email_ids` 和按完整公司名称键的 `disqualified` 原因代码映射。

采取行动前，请务必仔细阅读相关邮件——很多邮件看起来都很相似。

比如这个题，任务的主角Dana是一名采购经理。

她离开几天后回来，邮箱里已经堆了大约300封邮件。

但是呢，她有一个铝合金笔记本支架项目，必须在当天完成推进。

模型需要从这些邮件里还原项目相关的需求、最新的规格和数量，从20家供应商里找出真正合格、单件落地成本最低的一家。

选好供应商以后，它还需要进行一系列的规范化操作，比如只给最终选中的那封报价打标签，标出3封要求更换收款账户的可疑邮件，保存给供应商的回复草稿，创建启动会议，并交付一份规定格式的JSON总结。

这个任务量你想一想都知道有多大，而且这些任务的原始数据也非常乱，充分体现了现在人类的混乱性。

比如项目的最终需求零零碎碎的散在10封邮件里面，中途还改过数量，改过要求，改的乱七八糟的。

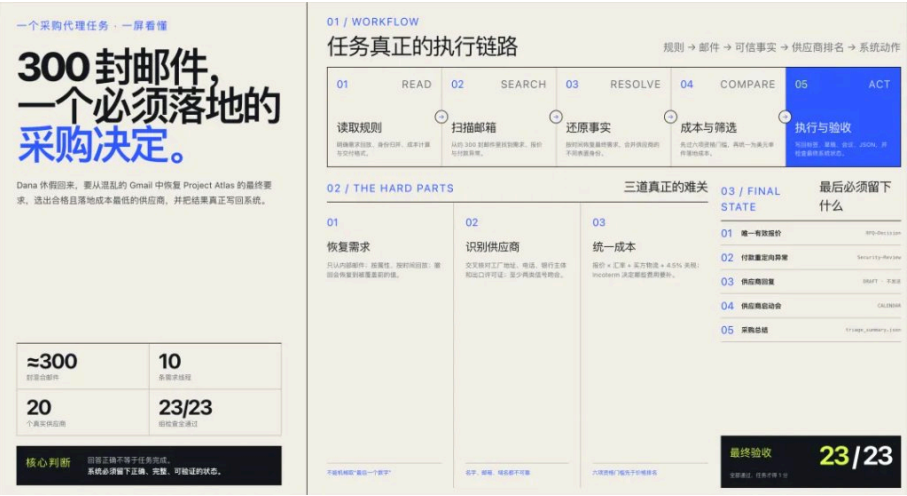
而且供应商也不是只看公司名，因为现实世界中，也经常会有同一家供应商会换着联系人、邮箱、域名和显示名称来发报价，所以题目中，同时还混进了4组名字很像、实际却是不同公司的供应商。

模型得综合工厂地址、电话、银行收款人和出口许可证等多个信号交叉核对，才能判断哪些报价来自同一家。

说实话，这个过程，看得我都要力竭了，至少我是一丁点都不想干的状态。。。



所以我也生了个页面，方便看得更清晰。



Accio Work团队那边，也给模型准备好了Shopify、Gmail、日历、商品发布之类的工具，可以让模型直接用。

等模型做完，评测程序会直接检查邮箱、草稿、日历和JSON文件的最终状态。

一共有23组、42项结果检查。



像选中的报价有没有加上正确的标签？3封钓鱼邮件有没有全部标记？草稿收件地址和会议日期对不对？JSON里的供应商、价格、数量和淘汰原因能不能全部对上？

而模型的得分，要这23组全部通过，才算答对，才能得到这1分，一旦有1个检验出来错误就0分，非常严格。

也就是说，没有过程分这个东西，你只有所有检测项全对才有分，因为模型去解决真实的电商问题的时候，这差一步，可能就会让你丢了单子，这才是真实的商业世界。

最后的得分就按通过的任务数除以总任务数量107。



于是，最终就是上文看到的排行榜，几乎没有及格的。

而如果用他们自己的Agent产品Accio的框架跑，通过率会整体高一些，模型通过率50%以上的，从2个增加到了6个，毕竟是专门做电商的框架。而因为是评测集，同时因为也是全自动化，平时大量的任务，其实是人和Agent协同，所以完成率也会实际更高一些。

Accio 13 个模型

#	模型	通过率	通过数	每任务成本	每任务耗时
1	Claude Opus 5	61.68%	66 / 107	\$3.50	10m 08s
2	Claude Opus 4.8	55.14%	59 / 107	\$4.17	11m 36s
3	Claude Opus 4.7	52.34%	56 / 107	\$3.30	6m 27s
4	Qwen 3.8 Max	52.34%	56 / 107	\$1.28	15m 42s
5	GPT-5.6 Sol	51.40%	55 / 107	\$2.79	5m 28s
6	DeepSeek V4 Pro	50.47%	54 / 107	\$0.238	14m 26s
7	DeepSeek V4 Flash	46.73%	50 / 107	\$0.043	10m 02s
8	Gemini 3.6 Flash	46.73%	50 / 107	\$1.34	4m 35s
9	GLM 5.2	46.73%	50 / 107	\$1.25	10m 49s
10	GPT-5.5	44.86%	48 / 107	\$2.00	4m 29s
11	GPT-5.6 Luna	44.86%	48 / 107	\$0.732	5m 40s
12	Gemini 3.6 Flash	43.88%	47 / 107	\$1.55	5m 44s
13	Gemini 3.6 Flash	43.88%	47 / 107	\$1.55	5m 44s

而如果我们对比一下Coding任务上的跑分，大家就能看出来差距了，我们拿今年5月底发布的DeepSWE举例。

这个评测基准包含了113个任务，考的是AI能不能在真实的开源代码仓库里，独立完成一次复杂、跨文件的软件工程改动。

113 个任务 · 每个任务都有一个稳定的 URL，包含说明、代码仓库和验证器

所有语言 113/113

加强模块加载、缓存自省和本地标志

加强 ABS 模块的解析和缓存。公开缓存自省 API，并使模块标志在脚本模式下正常工作。

abs-lang/abs 去

为数组和字符串添加进阶切片

为数组和字符串添加逐步切片解析、切片和赋值功能，并支持跨文件正确的索引。

abs-lang/abs 去

为操作和可重用工作流添加操作固定检查

添加可配置的 lint 规则，强制执行操作和可重用工作流引用的固定版本。

rhysd/actionlint 去

将输入键别名添加到名称映射

为名称映射添加仅加载别名支持，以便字段可以从备用输入键解析。

试用/adaptix PYTHON 去

向 aiomonitor 添加任务快照、检查和差异比较...

添加具有交互式检查、差异比较、删除和 Web/CLI 访问功能的即时任务快照。

aiio-lib/aiomonitor PYTHON 去

为 Anko 函数参数添加默认参数

在 Anko 函数中添加对默认参数的解析和调用时评估。

mattn/anko 去

向 Anko 添加类型化变量绑定

在 Anko 中添加类型化的 "var" 声明，并在启用 TypedBindings 时强制执行赋值时声明的类型约束。

mattn/anko 去

增加漂移检测和合规性基线

对容器配置实施基线捕获、漂移比较和合规性跟踪。

getarcanapp/arcan 去

添加 JSON Schema 引用和依赖关键字

添加 JSON Schema 依赖关键字，本地 \$defs/\$ref 解析和条件模式处理。

arktypeio/arktype 类型脚本 去

所有模型都用mini-swe-agent框架跑，模型自己去阅读代码仓库、查找问题、修改文件、运行测试，最后提交结果。

排在最前面的这几个模型，任务通过率都能到70%以上。

MODEL	PASS@1	AVG COST	OUT TOK	STEPS
claude-opus-5 [max]	74% ±4%	\$11.84	118k	99
gpt-5.6-sol [max]	73% ±3%	\$8.39	60k	61
claude-fable-5 [max]	70% ±4%	\$21.63	119k	88
kimi-k3 [max]	69% ±5%	\$4.65	81k	98
gpt-5.6-luna [max]	67% ±4%	\$0.61	73k	102
gpt-5.5 [xhigh]	67% ±6%	\$7.23	46k	82
grok-4.6 [xhigh]	67% ±2%	\$5.50	71k	87
gemini-3.7-flash [high]	65% ±2%	\$2.18	107k	125
deepseek-v4-pro [max]	63% ±6%	\$0.06	106k	155

类似的低分情况，也出现在了其他领域的真实任务评测集里。

比如上个月底，腾讯混元联合清华大学和东南大学发布的这个E-Bench。



2026-7-28



Bench: Benchmarking Multi-Step Tool-Use Agents in Real-World Product Scenarios

Wei Huang Zheng¹ Tianyuan Zou^{2,1} Eileen Ye¹ Alphet Liu¹ Youyong Kong³

Ya-Qin Zhang² Duran Zheng¹ Maxm Pan¹

¹Hunyuan Team, Tencent; ²Institute for AI Industry Research, Tsinghua University;
³School of Computer Science and Engineering, Southeast University

它模拟了3个产品场景，分别是王者荣耀、QQ音乐和腾讯会议。

一共设置了323个需要模型真实去操作的任务。比如王者荣耀要整理好友、战队、房间和比赛记录，QQ音乐要搜索歌曲、管理收藏歌单，腾讯会议要筛选参会人员、预定会议等。

E-Bench一共测了11个模型。

	Model	E-Bench		
		Avg@3	Pass@3	Pass ³
1	Kimi-K3	73.79%	87.62%	58.82%
2	GPT-5.5	72.03%	82.97%	57.59%
3	Opus-4.8	68.78%	84.33%	50.81%
4	Grok-4.5	66.10%	80.50%	52.32%
5	GLM-5.2	52.32%	71.52%	30.96%
6	Qwen-3.7-Max	50.88%	70.59%	30.34%
7	Hy3	50.15%	74.30%	26.63%
8	Seed-2.1-Pro	47.94%	65.33%	28.79%
9	Gemini-3.5-Flash	42.62%	64.71%	21.98%
10	MiniMax-M3	41.07%	62.85%	20.12%
11	DeepSeek-V4-Pro	34.47%	53.56%	17.34%

成绩最好的是Kimi-K3，Avg@3是73.79%。这里简单解释一下Avg@3这个指标，它指的是模型在任务集上独立运行3次，3次得分的平均值。

但现实中，我们肯定希望模型不只是做对一次，而是能反复多次完成任务。

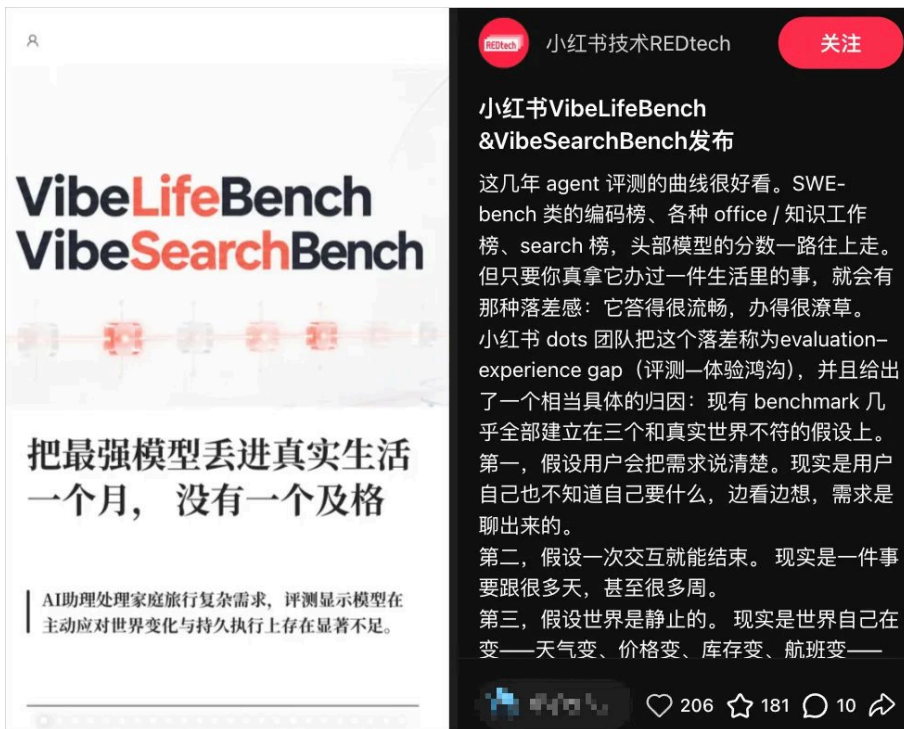
所以他们还测试了同一道题独立运行3次的情况。

表现最好的K33次全部做对只有58.82%。

而11个模型在这个稳定性指标上都没过60%。。。

还有比如小红书今年先后也发布了两套模型在真实任务上的评测。





比如这个偏生活的VibeLifeBench。

这个评测包含200个持续时间数周的生活任务，覆盖了职业、健身、租房、购物和差旅等10个领域。

同样还是给大家看看各个模型在榜单上的得分。

#	MODEL	AVG@3	MAX@3	MIN@3	Σ	INPUT (M)	OUTPUT	TOOL CALLS	TURNS
1	Opus-5	0.325	0.412	0.238	0.098	25.6	325,198	316	210
2	gpt-5.5	0.301	0.388	0.215	0.100	17.5	78,631	333	146
3	gemini-3.5-flash	0.275	0.356	0.201	0.083	41.2	213,757	243	227
4	opus-4.8	0.275	0.343	0.203	0.075	27.0	220,795	228	111
5	GLM 5.2	0.254	0.299	0.209	0.048	22.3	133,285	288	141
6	kimi-k2.6	0.226	0.271	0.184	0.046	21.7	120,516	231	166
7	deepseek-v4-pro	0.211	0.247	0.177	0.037	13.7	91,088	203	101

榜单上成绩最好的是Claude Opus 5，它的avg@3也依然只有32.5%。

就真的更有点惨不忍睹了。。。

这3套评测基准的任务、指标等等都不一样，所以不能横向比较，但至少能说明，在它们覆盖领域的真实世界任务上，大模型的表现，确实还有极大进步的空间。

所以这个时候，纯靠模型公司我觉得也不行了，需要所有产商一起努力，就像Accio也说，会持续提炼真实的工作任务，然后把这些评测集数据滚动更新然后开放。毕竟这些厂子最重要的目标，是做Agent，是做模型路由，只有模型牛逼了，他们才能实现给自己客户的价值最大化。

不过比较可惜的是，就是这些评测集更偏向于研究，更新的不够及时，一些新模型出来可能要很久以后才会更新，所以还没有办法放到AIHOT的排行榜里面。

不过我未来可能会把类似于阿里国际站的RealReplicaBench、腾讯的E-Bench、小红书的VibeLifeBench等等的真实世界工作的评测集全收集起来，然后我自己搭环境，我自己花钱来



跑，一发新模型就全部跑一遍，保持更新，再把这些评分放到AIHOT上，方便大家查看，大家可以期待一下～

大家如果有类似真实世界工作任务的评测集，也欢迎评论区留言跟我提供。

最后，我想说。

在AI的世界里，会解题，和会工作，中间确实还隔着一些比较遥远的距离。

模型在Coding这个任务上，现在确实做的很好，跑的很快，但是在大量真实工作任务的场景中，还有很大的进步空间。

毕竟，那些琐碎、混乱、没有标准答案的东西，才组成了我们每天真正面对的工作。

因为，这才是真正的。

人类世界。

本文来源：[数字生命卡兹克](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码

VIP会员·7天畅读卡 | 大师课·入门串讲

免费领取

限免

45 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论

1 条评论

撒斯给 VIP会员

32分钟前 中国

不懂其他行业，我们公司属于制造业，大量使用AI

0 回复

